Security Audit Report

Gno Sprint 1

v1.0

January 17, 2025

Table of Contents

Table of Contents 2
License 4
Disclaimer 5
Introduction 6
Purpose of This Report 6
Codebase Submitted for the Audit 6
Methodology 8
Functionality Overview 8
How to Read This Report 9
Code Quality Criteria 10
Summary of Findings 11
Detailed Findings 13
1. Attackers can craft malicious contracts and trigger a denial of service 13
2. Attackers can abuse source code formatting to trigger a denial of service 14
3. Lack of call stack depth limit could introduce non-determinism during machine execution 15
4. Unbounded resource usage in the queryEval function leads to a denial of service 15
5. Genesis signature verification is disabled in production environments 16
6. Potential memory overflow during package loading 16
7. Incorrect AVL tree balance condition causes degraded performance 17
8. Validators can be registered with unrelated addresses and public keys 17
9. Potential precision loss during YayPercent calculation 18
10. Lack of validation allows the submission of empty files 18
11. Potential underflow during AVL node height calculation 19
12. Centralization risks in call verification logic 19
13. Distinct user inputs are incorrectly parsed into the same Gno values 20
14. Machine-dependent integer parsing may cause inconsistent behavior across different architectures 20
15. Unsanitized outputs may lead to client-side attacks 21
16. Potential chain halt due to unhandled panics in EndBlocker 22
17. Redundant coin transfer operation 23
18. Mutable chain domain increases validation complexity 23
19. Regular expressions in use are vulnerable to denial of service 24
20. Redundant field in MsgRun 25
21. Suboptimal developer experience when iterating over collections 25
22. Missing Profile length validation 26
23. Potentially confusing index value for non-existent keys 26
24. Lack of pagination in the GetValidators function may lead to gas exhaustion 27
25. Missing Bech32 address validation for the invite key 27

26. Lack of warning for skipFailingGenesisTxs during production 28
27. Suboptimal AST generation in the MsgCall handler 28
28. Usage of magic numbers decreases maintainability 29
29. Contracts should implement a two-step ownership transfer 29
30. Outdated comments 30
31. Unimplemented functionalities in the codebase 30
32. Panics in the VM affect node execution 31
33. Outstanding TODO and XXX comments 31
Potential attack vectors and considerations **33**
Execution 33
State 33
Memory management 34
Contracts 34
Crypto 35
Dependencies 35
Appendix **36**
1. Example source code for “Attackers can craft malicious contracts and trigger a denial of service” 36
2. Example scenario for “Attackers can abuse source code formatting to trigger a denial of service” 36
3. Proof of concept for “Attackers can abuse source code formatting to trigger a denial of service” 37
4. Proof of concept for “Potential underflow during AVL node height calculation” 39

License

THIS WORK IS LICENSED UNDER A CREATIVE COMMONS ATTRIBUTION-NODERIVATIVES 4.0 INTERNATIONAL LICENSE.

Disclaimer

THE CONTENT OF THIS AUDIT REPORT IS PROVIDED “AS IS”, WITHOUT REPRESENTATIONS AND WARRANTIES OF ANY KIND.

THE AUTHOR AND HIS EMPLOYER DISCLAIM ANY LIABILITY FOR DAMAGE ARISING OUT OF, OR IN CONNECTION WITH, THIS AUDIT REPORT.

THIS AUDIT REPORT WAS PREPARED EXCLUSIVELY FOR AND IN THE INTEREST OF THE CLIENT AND SHALL NOT CONSTRUE ANY LEGAL RELATIONSHIP TOWARDS THIRD PARTIES. IN PARTICULAR, THE AUTHOR AND HIS EMPLOYER UNDERTAKE NO LIABILITY OR RESPONSIBILITY TOWARDS THIRD PARTIES AND PROVIDE NO WARRANTIES REGARDING THE FACTUAL ACCURACY OR COMPLETENESS OF THE AUDIT REPORT.

FOR THE AVOIDANCE OF DOUBT, NOTHING CONTAINED IN THIS AUDIT REPORT SHALL BE CONSTRUED TO IMPOSE ADDITIONAL OBLIGATIONS ON COMPANY, INCLUDING WITHOUT LIMITATION WARRANTIES OR LIABILITIES.

COPYRIGHT OF THIS REPORT REMAINS WITH THE AUTHOR.

This audit has been performed by

Oak Security GmbH

https://oaksecurity.io/
info@oaksecurity.io

Introduction

Purpose of This Report

Oak Security GmbH has been engaged by NewTendermint LLC to perform a security audit of Gno.

The objectives of the audit are as follows:

1. Determine the correct functioning of the protocol, in accordance with the project specification.
2. Determine possible vulnerabilities, which could be exploited by an attacker.
3. Determine smart contract bugs, which might lead to unexpected behavior.
4. Analyze whether best practices have been applied during development.
5. Make recommendations to improve code safety and readability.

This report represents a summary of the findings.

As with any code audit, there is a limit to which vulnerabilities can be found, and unexpected execution paths may still be possible. The author of this report does not guarantee complete coverage (see disclaimer).

Codebase Submitted for the Audit

The audit has been performed on the following target:

Repository	https://github.com/gnolang/gno
Commit	`88fb8cb3aac1e3b393a9f8e88304b78fbe1240da`
Scope	The audit scope focuses on the interactions between components, smart contract injection risks, and specific example packages and realms within the GnoLand ecosystem. ● Interactions ○ Assessment of the interaction between smart contracts (realms) and the GnoVM, focusing on the implementation of the VM Keeper.

- Verification of operations executed at the end of blocks.
- Assessment of the correctness and security of processes that update validator states.

- Smart contract injection risks
  - Assessment of mechanisms for deploying and executing smart contracts (realms). This includes the review of the following message handlers:
    - `MsgAddPkg`
    - `MsgCall`
    - `MsgRun`
- Example realms and packages
  - The audit includes specific examples of realms and packages to assess their security implications and identify vulnerabilities. The examples reviewed are:
    - `./examples/gno.land/p/sys/validators`
    - `./examples/gno.land/p/nt/poa`
    - `./examples/gno.land/p/demo/avl`
    - `./examples/gno.land/p/demo/ownable`
    - `./examples/gno.land/r/sys/params`
    - `./examples/gno.land/r/sys/users`
    - `./examples/gno.land/r/sys/validators`

Methodology

The audit has been performed in the following steps:

1. Gaining an understanding of the code base's intended purpose by reading the available documentation.
2. Automated source code and dependency analysis.
3. Manual line-by-line analysis of the source code for security vulnerabilities and use of best practice guidelines, including but not limited to:
    a. Race condition analysis
    b. Under-/overflow issues
    c. Key management vulnerabilities
4. Report preparation

Functionality Overview

Gno is an interpreted and fully-deterministic implementation of the Go programming language, designed to build succinct and composable smart contracts.

The first blockchain to use it is gno.land, a Proof of Contribution-based chain, backed by a variation of the Tendermint consensus engine.

How to Read This Report

This report classifies the issues found into the following severity categories:

Severity	Description
Critical	A serious and exploitable vulnerability that can lead to loss of funds, unrecoverable locked funds, or catastrophic denial of service.
Major	A vulnerability or bug that can affect the correct functioning of the system, lead to incorrect states or denial of service.
Minor	A violation of common best practices or incorrect usage of primitives, which may not currently have a major impact on security, but may do so in the future or introduce inefficiencies.
Informational	Comments and recommendations of design decisions or potential optimizations, that are not relevant to security. Their application may improve aspects, such as user experience or readability, but is not strictly necessary. This category may also include opinionated recommendations that the project team might not share.

The status of an issue can be one of the following: **Pending**, **Acknowledged**, **Partially Resolved**, or **Resolved**.

Note that audits are an important step to improving the security of smart contracts and can find many issues. However, auditing complex codebases has its limits and a remaining risk is present (see disclaimer).

Users of the system should exercise caution. In order to help with the evaluation of the remaining risk, we provide a measure of the following key indicators: **code complexity**, **code readability**, **level of documentation**, and **test coverage**. We include a table with these criteria below.

Note that high complexity or low test coverage does not necessarily equate to a higher risk, although certain bugs are more easily detected in unit testing than in a security audit and vice versa.

Code Quality Criteria

The auditor team assesses the codebase's code quality criteria as follows:

Criteria	Status	Comment
Code complexity	**High**	The codebase in scope includes the interaction of multiple components, including the gno.land chain, the GnoVM, and Tendermint. The chain incorporates unique concepts and while smart contracts are written in Gno, a new language derived from Go.
Code readability and clarity	**Medium-High**	Numerous unresolved TODOs have been found in the codebase.
Level of documentation	**Medium-High**	The provided documentation is extensive.
Test coverage	**Medium-High**	Test coverage reported by `go test` for `gno.land/pkg` is `66.70%`. Gno smart contracts do not provide tooling for computing test coverage.

Summary of Findings

No	Description	Severity	Status
1	Attackers can craft malicious contracts and trigger a denial of service	Critical	Reported
2	Attackers can abuse source code formatting to trigger a denial of service	Critical	Reported
3	Lack of call stack depth limit could introduce non-determinism during machine execution	Critical	Reported
4	Unbounded resource usage in the `queryEval` function leads to a denial of service	Major	Reported
5	Genesis signature verification is disabled in production environments	Major	Reported
6	Potential memory overflow during package loading	Major	Reported
7	Incorrect AVL tree balance condition causes degraded performance	Major	Reported
8	Validators can be registered with unrelated addresses and public keys	Major	Reported
9	Potential precision loss during `YayPercent` calculation	Major	Reported
10	Lack of validation allows the submission of empty files	Minor	Reported
11	Potential underflow during AVL node height calculation	Minor	Reported
12	Centralization risks in call verification logic	Minor	Reported
13	Distinct user inputs are incorrectly parsed into the same Gno values	Minor	Reported
14	Machine-dependent integer parsing may cause inconsistent behavior across different architectures	Minor	Reported
15	Unsanitized outputs may lead to client-side attacks	Minor	Reported
16	Potential chain halt due to unhandled panics in `EndBlocker`	Minor	Reported
17	Redundant coin transfer operation	Informational	Reported

18	Mutable chain domain increases validation complexity	**Informational**	Reported
19	Regular expressions in use are vulnerable to denial of service	**Informational**	Reported
20	Redundant field in `MsgRun`	**Informational**	Reported
21	Suboptimal developer experience when iterating over collections	**Informational**	Reported
22	Missing `Profile` length validation	**Informational**	Reported
23	Potentially confusing index value for non-existent keys	**Informational**	Reported
24	Lack of pagination in the `GetValidators` function may lead to gas exhaustion	**Informational**	Reported
25	Missing `Bech32` address validation for the `invite` key	**Informational**	Reported
26	Lack of warning for `skipFailingGenesisTxs` during production	**Informational**	Reported
27	Suboptimal AST generation in the `MsgCall` handler	**Informational**	Reported
28	Usage of magic numbers decreases maintainability	**Informational**	Reported
29	Contracts should implement a two-step ownership transfer	**Informational**	Reported
30	Outdated comments	**Informational**	Reported
31	Unimplemented functionalities in the codebase	**Informational**	Reported
32	Panics in the VM affect node execution	**Informational**	Reported
33	Outstanding `TODO` and `XXX` comments	**Informational**	Reported

Detailed Findings

1. Attackers can craft malicious contracts and trigger a denial of service

Severity: Critical

In `gno.land/pkg/sdk/vm/keeper.go`, the `MsgAddPackage` message allows users to add new packages to the VM. This process involves the execution of the `AddPackage` function, which eventually invokes the `TypeCheckMemPackage` function in line `352`. The `TypeCheckMemPackage` function utilizes Go's `parseCheckMemPackage` for parsing and type-checking source files to generate abstract syntax trees (AST).

However, since this operation is executed in memory, no additional gas is metered or incurred. This allows malicious actors to submit syntactically valid but deeply nested code structures to perform a denial of service attack. Deeply nested expressions can force the parser and type-checker to descend multiple levels, non-linearly increasing computational complexity and resource usage.

While the parser implements a `maxNestLev` nesting depth limit of `100_000`, this threshold is considered significantly high, allowing excessive resource exhaustion to occur before reaching the depth limit. Additionally, attackers can craft a malicious source code to build up and clear the stack multiple times to bypass the limitation.

Consequently, this asymmetry allows attackers to pay gas proportionally only to source code file size while inflicting disproportionate computational costs, resulting in substantial CPU and memory resource consumption, ultimately leading to denial of service.

A minimal example of a malicious source code is provided in Appendix 1.

This issue is also present in the `MsgRun` message because it internally calls the `TypeCheckMemPackage` function.

Recommendation

We recommend introducing gas metering for parsing and type-checking operations to incur gas costs based on the computation power consumed. Additionally, consider decreasing the maximum allowed nesting level and enforcing stricter resource usage caps during parsing and type-checking processes.

Status: Reported

This audit was performed on an early version of the protocol.

2. Attackers can abuse source code formatting to trigger a denial of service

Severity: Critical

In `gno.land/pkg/sdk/vm/keeper.go`, during the execution of the `MsgAddPackage` message, the `parseCheckMemPackage` function is invoked with the `fmt` parameter set to `true`. This causes the `format.Node` function in `gnovm/pkg/gnolang/go2gno.go:616` to be called, which formats the Go source file into its canonical form with `go/format` (commonly referred to as `gofmt`).

The applied formatting adheres to standard Go style rules, such as:

- Adding newlines before declarations, if statements, for loops, etc.
- Adjusting indentation within blocks.
- Ensuring one statement per line (excluding specific cases, like single-line variable declarations).
- Improve clarity by adding newlines and sorting imports for full files.

After the formatting has been applied, the file size will increase and differ from the original file submitted.

The issue is that gas is only charged based on the initially submitted file size during the `anteHandler` phase. The difference between the initial file size and the post-formatted file size is not accounted for, allowing users to store a file disproportionally larger than the incurred gas cost.

Consequently, this discrepancy enables attackers to craft and submit a small malicious file that significantly expands after the formatting is automatically applied. This creates an asymmetry between the required gas cost and actual resource power consumed, leading to a denial of service attack.

A minimal example of malicious source code and proof of concept is provided in Appendix 2 and Appendix 3.

Recommendation

We recommend modifying the implementation to enforce that files must be submitted in their canonical format and return an error if the output from on-chain formatting differs from the provided file. This would prevent users from paying gas for a small file and then maliciously expanding it without incurring additional gas costs.

Status: Reported

This audit was performed on an early version of the protocol.

3. Lack of call stack depth limit could introduce non-determinism during machine execution

Severity: Critical

When initializing the virtual machine in `gno.land/pkg/sdk/vm/keeper.go`, the `NewMachineWithOptions` function does not implement any maximum call stack depth limits. The current behavior of the machine relies solely on the gas meter to regulate execution, which allows call stack depths to grow dynamically without any limitation.

The issue is that depending on the underlying machine's memory capacity, the dynamic stack size could create unexpected crashes in some nodes while others continue to operate. This is because nodes with a lesser memory capacity are more likely to run out of memory than nodes with a larger memory capacity. For example, a node with 8GB of RAM could process deep call stacks, but a node with 2GB of RAM could have crashed due to a stack overflow.

Consequently, machines with different memory capacities may experience different behaviors when processing the same package, causing non-determinism to occur and ultimately leading to a chain halt.

Recommendation

We recommend introducing a standardized maximum call stack depth limit to ensure all nodes behave deterministically regardless of their memory capacity.

Status: Reported

This audit was performed on an early version of the protocol.

4. Unbounded resource usage in the `queryEval` function leads to a denial of service

Severity: Major

In `gno.land/pkg/sdk/vm/handler.go:152-180`, the `queryEval` function evaluates arbitrary expressions provided by users without restricting the resource usage limits.

Specifically, the `queryEval` function uses `infiniteGasMeter` throughout the query execution. This means there is no upper bound limit on resource usage, including CPU usage and memory consumption.

Consequently, attackers could specify complex expressions with exponential time complexity to overload nodes, ultimately crashing them and causing a denial of service.

Recommendation

We recommend enforcing a maximum gas limit for expressions evaluated by `queryEval`.

Status: Reported

This audit was performed on an early version of the protocol.

5. Genesis signature verification is disabled in production environments

Severity: Major

In `gno.land/pkg/gnoland/app.go:108`, the `VerifyGenesisSignatures` parameter is purposely set to `false` for development purposes. This bypasses the verification of signatures during genesis processing, which is acceptable in a testing environment.

However, this poses significant security risks in production environments. Skipping the verification of genesis signatures allows invalid or malicious genesis states to be accepted, compromising the integrity of the chain.

Recommendation

We recommend setting `VerifyGenesisSignatures` to `true` in production environments to ensure only valid and verified signatures are accepted during genesis.

Status: Reported

This audit was performed on an early version of the protocol.

6. Potential memory overflow during package loading

Severity: Major

When loading packages in `gno.land/pkg/sdk/vm/keeper.go:103`, the `Allocator` is configured with a `maxAllocTx` value of `500MB` to allocate memory for `baseStore` and `iavlStore`. At the same time, a cache consisting of (`name`, `types`) grows dynamically as packages are processed.

The issue is that deploying a large number of packages (e.g., millions of realms) could cause memory exhaustion during node initialization. As the cache expands, memory usage increases beyond what `maxAllocTx` alone can accommodate. Since `maxAllocTx` is a fixed allocation and does not scale dynamically with package loading, nodes may run out of memory and crash before completing initialization.

Consequently, nodes may fail to start in large-scale deployments when extensive packages are loaded during startup.

Recommendation

We recommend implementing lazy package loading. Instead of loading all packages into memory during startup, they should be loaded on demand.

Status: Reported

This audit was performed on an early version of the protocol.

7. Incorrect AVL tree balance condition causes degraded performance

Severity: Major

In `examples/gno.land/p/demo/avl/node.gno:285-313`, the `balance` function is responsible for rebalancing AVL trees. The function returns early if the tree is considered balanced, based on the condition `balance >= -1`.

However, this condition is incorrect and should be replaced with `balance >= -1 && balance <= 1`. As a result, the subsequent `if` statement that checks `balance > 1` will never be reached because the `balance > 1` condition is already covered by the `balance >= -1` condition. This prevents the `balance` function from performing a rebalance in cases where the left subtree is significantly heavier than the right subtree.

Consequently, since AVL trees rely on self-balancing to maintain optimal performance, failing to trigger rebalancing causes inefficient data access and degraded execution speed. Given that AVL trees are the core building block for almost all collections within the virtual machine, this inefficiency leads to inflated gas fees for users when executing complex smart contracts.

Recommendation

We recommend correcting the condition to `balance >= -1 && balance <= 1` to ensure proper rebalancing.

Status: Reported

This audit was performed on an early version of the protocol.

8. Validators can be registered with unrelated addresses and public keys

Severity: Major

In `examples/gno.land/p/nt/poa/poa.gno`, the `AddValidator` function of the `PoA` package allows the caller to specify both an address and a public key when creating a validator.

However, the function does not verify that the provided address is derived from the corresponding public key, allowing users to create a `Validator` instance where the address and public key are entirely unrelated.

Consequently, this inconsistency can undermine the integrity of the validator set.

Recommendation

We recommend modifying the `AddValidator` function to validate that the address is derived from the provided public key.

Status: Reported

This audit was performed on an early version of the protocol.

9. Potential precision loss during `YayPercent` calculation

Severity: Major

In `examples/gno.land/p/demo/dao/vote.gno`, the `YayPercent` function uses integer division to compute the percentage of "`yay`" votes in relation to the total voting power.

However, this approach is problematic because it may result in precision loss, particularly in edge cases where the total voting power is small. In some situations, all "`yay`" votes may be effectively discarded, causing the function to incorrectly return `0%` even when "`yay`" votes exist.

For example, if `v.YayVotes` is `1` and `v.TotalVotingPower` is `200`, the actual percentage should be `0.5%`. However, due to integer truncation, the function returns `0%`.

Consequently, this leads to proposals being incorrectly interpreted as having no support even if "`yay`" votes are present, causing a critical misrepresentation of voting outcomes.

Recommendation

We recommend enhancing precision in the aforementioned calculation.

Status: Reported

This audit was performed on an early version of the protocol.

10. Lack of validation allows the submission of empty files

Severity: Minor

The `Validate` function of `MemPackage` is defined in `gnovm/memfile.go:51-91`, and it performs various checks to ensure the correctness of the provided data.

However, it does not validate the `Body` field of the provided files. Specifically, the `Body` field is not validated to be a non-empty string, allowing files without content to be submitted.

Recommendation

We recommend enhancing the `Validate` function of `MemPackage` to validate that the `Body` field is not empty.

Status: Reported

This audit was performed on an early version of the protocol.

11. Potential underflow during AVL node height calculation

Severity: Minor

In `examples/gno.land/p/demo/avl/node.gno:276`, the `height` calculation may result in an underflow. When the maximum value of `int8` (which is `127`) is exceeded, incrementing it by `1` causes the height to wrap to `-128`. Since this does not trigger a panic or revert, the height value will be corrupted silently.

Additionally, the likelihood of the `height` variable reaching the maximum value may happen due to a bug in the balancing logic, making the issue more prevalent.

A minimal example of proof of concept is provided in Appendix 4.

Recommendation

We recommend implementing validations to detect and handle overflow conditions.

Status: Reported

This audit was performed on an early version of the protocol.

12. Centralization risks in call verification logic

Severity: Minor

In `examples/gno.land/r/sys/users/verify.gno:73`, the admin has the ability to specify any function as the call verification logic. Since nearly all interactions with a smart contract platform involve function calls, this level of control grants excessive privileges to the admin, introducing a centralization risk.

We classify this issue as minor severity since only admins, who are privileged addresses, have the authority to modify the call verification logic.

Recommendation

We recommend limiting the reliance on admin roles for access control by adopting a more restrictive approach, such as a whitelist, where users can be added or removed on a per-namespace basis.

Status: Reported

This audit was performed on an early version of the protocol.

13. Distinct user inputs are incorrectly parsed into the same Gno values

Severity: Minor

In `gno.land/pkg/sdk/vm/convert.go`, conversions from `String` values to properly typed Gno values are implemented. While efforts have been made to prevent distinct `String` values from being interpreted as the same Gno values, some cases are not covered:

- Lines `13-17`: the `assertNoPlusPrefix` function processes numerical inputs and rejects redundant prefixes (e.g., "`+`" in "`+1`"), allowing only canonical representations such as "`1`". However, it does not handle the case where the input is "`-0`", resulting in both "`0`" and "`-0`" being converted into zero.
- Line `202`: the `strconv.ParseFloat` function is used to convert numerical strings into floating-point values. However, this function ignores character case, treating the following inputs as identical:
  - "`NaN`" and "`nan`"
  - "`Inf`" and "`inf`"
  - "`-Inf`" and "`-inf`"

Recommendation

We recommend enforcing case-sensitive validation for converted strings and explicitly rejecting "`-0`".

Status: Reported

This audit was performed on an early version of the protocol.

14. Machine-dependent integer parsing may cause inconsistent behavior across different architectures

Severity: Minor

In `gno.land/pkg/sdk/vm/convert.go`, the `convertArgToGno` function parses user-provided `String` values into machine-dependent types, specifically `int` and `uint`. In line `46`, the input is first parsed into an `i64` variable before being coerced into `int` at line `52`.

Similarly, in line `96`, the input is parsed into a `u64` variable before being coerced into `uint` at line `102`.

However, the function does not restrict 32-bit machines from participating in the network. On such machines, coercion occurs in a 32-bit type. If the `String` value represents a number that exceeds the 32-bit range, it will be interpreted differently across architectures, leading to inconsistencies.

Recommendation

We recommend either preventing 32-bit machines from participating in the network or removing support for machine-dependent types.

Status: Reported

This audit was performed on an early version of the protocol.

15. Unsanitized outputs may lead to client-side attacks

Severity: Minor

In `gno.land/pkg/sdk/vm/handler.go:120-137`, the `queryRender` function allows users to execute the `Render` function of a specified package, producing a string intended to represent the package state in a human-readable format. Since there is no standard output format, some packages return markdown, while others produce plain HTML.

However, the function does not sanitize the returned output, allowing malicious packages to include scripts in the output to trigger a client-side attack.

For example, malicious scripts could execute arbitrary code in HTML outputs, leading to Cross-Site Scripting (XSS) attacks or phishing attempts. Additionally, if a markdown output contains embedded HTML tags, malicious scripts could also be executed depending on the client's implementation.

Recommendation

We recommend documenting secure practices for users and developers when processing `queryRender` outputs in their clients.

Alternatively, a stricter approach would involve implementing sanitization mechanisms. An example of HTML outputs would be to strip or neutralize potentially dangerous tags (e.g., `<script>`, `<iframe>`) and attributes (e.g., `onload`, `onclick`).

Status: Reported

This audit was performed on an early version of the protocol.

16. Potential chain halt due to unhandled panics in `EndBlocker`

Severity: Minor

In `gno.land/pkg/gnoland/app.go:379-431`, the `EndBlocker` function is called at the end of every block. Since this function does not enforce gas limits during computation and does not tolerate panics, excessive computation or an unhandled panic would result in a chain halt. Hence, careful review should be performed to avoid halting the chain unintendedly.

Below are instances where a potential chain halt may occur:

- In lines `407-412`, a query to the `gno.land/r/sys/validators/v2` realm is executed with the `vmk.QueryEval` function. This function is defined in `gno.land/pkg/sdk/vm/keeper.go`, which is responsible for evaluating arbitrary Gno expressions in the context of a specific realm. In this case, its usage is to extract validator updates.

  The `QueryEval` function invokes the `doRecover` function in lines `531-540` to recover from potential panics. However, the `doRecover` function does not handle panics for out-of-gas errors as it explicitly rethrows the error on line `522`.

  Consequently, if an out-of-gas error occurs, the chain will be halted.

- The output of the `gno.land/r/sys/validators/v2` realm is evaluated using a hardcoded regular expression. This regular expression is defined in `gno.land/pkg/gnoland/validators.go:20` as below:

  ```
  {\("([^"]*)"\s[^)]+\),\("((?:[^"]|\\")*)"\s[^)]+\),\((\d+)\
  s[^)]+\)}
  ```

  The issue is that this regular expression can be exploited to cause denial of service attacks with the following malicious input designed to trigger excessive computation:

  ```
  }{(""\t}'.repeat(864) + '}),("\\' +
  '"\\"\\"\t\\'.repeat(864) + '0),(""\t"),(0\t,)}
  ```

  Consequently, if the malicious input is processed by the regular expression, excessive computation will be triggered, causing block production timeout and halting the chain.

We classify this issue as minor severity because the current version of the `sys/validators` realm is neither susceptible to running out of gas nor producing malicious output. Nevertheless, as the codebase evolves, this could become a more severe issue.

Recommendation

We recommend handling potential out-of-gas errors and denial of service issues in the `EndBlocker` function.

Status: Reported

This audit was performed on an early version of the protocol.

17. Redundant coin transfer operation

Severity: Informational

In `gno.land/pkg/sdk/vm/keeper.go:572`, the `SendCoins` function performs a fund transfer from the `caller` to `pkgAddr`.

However, the `pkgAddr` and `caller` refer to the same address in this context, making the operation redundant.

Consequently, since transferring coins to the same address does not alter the system state, this results in unnecessary computation and inefficiency.

Recommendation

We recommend removing the redundant `SendCoins` call.

Status: Reported

This audit was performed on an early version of the protocol.

18. Mutable chain domain increases validation complexity

Severity: Informational

In `gno.land/pkg/sdk/vm/params.go:7`, the on-chain `chainDomainParamPath` parameter is defined. This parameter allows governance to vote on a new chain domain, distinct from "`gno.land`".

While governance facilitates blockchain operations by eliminating the need for coordinated chain upgrades, the ability to change the chain domain introduces complexity in validation and requires careful planning for package and realm migrations. Additionally, the benefits of altering the chain domain remain unclear.

Below are a few examples of limited validation and increased complexity:

- Instead of a precise chain domain check, the `ValidateBasic` function for `MsgAddPackage` only checks that the path is not empty. A more efficient approach would be to move the prefix validation from `gno.land/pkg/sdk/vm/keeper.go:245` to `sdk/vm/msgs.go:57`. Additionally, the usage of regular expressions in `sdk/vm/keeper.go:346` is excessive. The ability to mutate the chain domain prevents simple trim of the constant prefix, followed by validations that check the path remainder starts with "`g`" and ends with "`/run`".

- The `ValidateBasic` function for `MsgCall` only validates that the path can be a realm path in `sdk/vm/msgs.go:120`. Additionally, this is performed using complex regular expressions rather than a simple trim of the constant prefix. The chain domain is also not explicitly validated in the `MsgCall` handler in `sdk/vm/keeper.go`. If the

path contains an invalid chain domain, it is only rejected later by the package management layer.

- The `ValidateBasic` function for `MsgRun` does not validate the chain domain nor check whether it contains only allowed characters or has the correct number of path elements (determined by counting "/" characters). Instead, it only validates the suffix, leaving potential inconsistencies unchecked.

Consequently, this design decision affects the performance of package path validation. Since nodes must retrieve the parameter value from the VM, the `ValidateBasic` functions cannot efficiently validate package paths without additional complexity.

Recommendation

We recommend defining the chain domain as a constant, a configuration parameter, or a genesis parameter. This constant should then be used in `ValidateBasic` functions to match package paths against the constant chain domain while validating others with less complex rules.

Status: Reported

This audit was performed on an early version of the protocol.

19. Regular expressions in use are vulnerable to denial of service

Severity: Informational

Throughout the codebase, numerous regular expressions are defined. Some of them are vulnerable to denial of service attacks:

- In `gno.land/pkg/gnoland/validators.go:20`, the following regular expression is defined:

  ```
  {\("([^"]*)"\s[^)]+\),\("((?:[^"]|\\")*)"\s[^)]+\),\((\d+)\
  s[^)]+\)}
  ```

  The input strings for this expression are generated by a predefined system realm.

- In `gnovm/memfile.go:44`, the following regular expression is defined:

  ```
  ^([a-zA-Z0-9-]+\.)*[a-zA-Z0-9-]+\.[a-zA-Z]{2,}\/(?:p|r)(?:\
  /_?[a-z]+[a-z0-9_]*)+$
  ```

  The input strings for this expression are generated from the message handlers in `gno.land/pkg/sdk/vm/keeper.go:337` and `561`.

Although we did not find an attack vector that could exploit these regular expressions to cause a denial of service attack, their complexity introduces maintenance challenges and

potential risks. If not addressed proactively, these risks could become more significant as the codebase evolves.

Recommendation

We recommend refactoring the identified expressions to minimize complexity where feasible, reducing the risk of denial of service vulnerabilities.

Status: Reported

This audit was performed on an early version of the protocol.

20. Redundant field in `MsgRun`

Severity: Informational

In `gno.land/pkg/sdk/vm/keeper.go:554`, the `memPkg.Path` is assigned a string derived from the chain domain and the caller's address. Previously, this variable was set to `msg.Package.Path`, but it was discarded.

However, `msg.Package.Path` is only used in the `ValidateBasic` function for partial validation, and since the chain domain inside the user-provided package path is not validated in `ValidateBasic`, this field becomes redundant.

Recommendation

We recommend removing the `msg.Package.Path` field.

Status: Reported

This audit was performed on an early version of the protocol.

21. Suboptimal developer experience when iterating over collections

Severity: Informational

In `examples/gno.land/p/demo/avl/node.gno`, the `TraverseInRange` function serves as a low-level building block for defining the more developer-friendly `Iterate` and `ReverseIterate` functions, which are implemented in lines `315-323`.

However, lines `326-327` contain a comment describing the iteration behavior of `TraverseInRange`:

> *"Start is inclusive and end is exclusive when ascending, start and end are inclusive when descending."*

Consequently, this inconsistency in iteration bounds may lead to unexpected differences in the number of elements visited when using `Iterate` and `ReverseIterate` on the same collection with identical bounds, potentially confusing developers.

Recommendation

We recommend ensuring that the API exposed to developers is consistent and predictable in its iteration behavior.

Status: Reported

This audit was performed on an early version of the protocol.

22. Missing `Profile` length validation

Severity: Informational

In `examples/gno.land/r/demo/users/users.gno:108`, the `Profile` field of the `User` struct does not enforce a length validation during initialization.

Consequently, this omission can lead to oversized profile data being stored, hindering performance.

Recommendation

We recommend implementing a validation mechanism for the `Profile` field to ensure that profiles adhere to size constraints to prevent potential misuse.

Status: Reported

This audit was performed on an early version of the protocol.

23. Potentially confusing index value for non-existent keys

Severity: Informational

In `examples/gno.land/p/demo/avl/node.gno`, the `Get` function returns three values for a given key: `index`, `value`, and a boolean flag `exists`. When a key is not found in the collection (lines `84` and `94`), the function sets `exists` to `false`.

However, the `index` is incorrectly set to `0`, which may unintentionally reference an existing item in the collection. This could lead to confusion or unintended behavior in functions that depend on the returned index.

Recommendation

We recommend setting `index` to a value that can never exist (e.g., `-1`) to prevent ambiguity.

Status: Reported

This audit was performed on an early version of the protocol.

24. Lack of pagination in the `GetValidators` function may lead to gas exhaustion

Severity: Informational

In `examples/gno.land/p/nt/poa/poa.gno:95-105`, the `GetValidators` function of the `PoA` package retrieves and returns all stored validators after iterating through them.

However, if the number of validators is substantial, this operation can consume a significant amount of gas, potentially exceeding gas limits and causing the transaction to fail.

Recommendation

We recommend implementing a pagination mechanism in the `GetValidators` function to allow the caller to retrieve validators by batches.

Status: Reported

This audit was performed on an early version of the protocol.

25. Missing `Bech32` address validation for the `invite` key

Severity: Informational

In `examples/gno.land/r/demo/users/users.gno:151`, the `line` variable is directly used to construct the `invitekey` without validating whether the address is a valid `Bech32` address.

Consequently, this could lead to potential issues where invalid or malformed addresses are recorded, causing downstream errors or inconsistencies.

Recommendation

We recommend implementing `Bech32` address validation for the `line` variable before constructing the `invitekey`.

Status: Reported

This audit was performed on an early version of the protocol.

26. Lack of warning for `skipFailingGenesisTxs` during production

Severity: Informational

In `gno.land/cmd/gnoland/start.go:239`, the `skipFailingGenesisTxs` flag is used to bypass a scenario where genesis transactions fail during node initialization.

While this is useful for testing purposes, starting the application with the `skipFailingGenesisTxs` flag set to `true` in production environments may compromise the integrity of the blockchain. For example, it may allow the chain to start with an incomplete or invalid genesis state, which could lead to unforeseen issues or vulnerabilities.

Recommendation

We recommend issuing a warning when the `skipFailingGenesisTxs` flag is enabled to inform users of potential risks outside of testing environments.

Status: Reported

This audit was performed on an early version of the protocol.

27. Suboptimal AST generation in the `MsgCall` handler

Severity: Informational

In `gno.land/pkg/sdk/vm/keeper.go:427-435`, the `Call` function constructs a function call template as a plain string with placeholders ("`arg1`", "`arg2`", etc.), then parses the string into an Abstract Syntax Tree (AST). Subsequently, in lines `452-458`, the function populates the `Args` slice of the AST with actual values.

However, this approach introduces unnecessary overhead by first creating a string and then parsing it rather than directly constructing the AST. As a result, the implementation is inefficient in terms of performance, robustness, and maintainability.

Recommendation

We recommend streamlining the generation of the expression to evaluate by generating the AST directly without the use of the Gno parser.

Status: Reported

This audit was performed on an early version of the protocol.

28. Usage of magic numbers decreases maintainability

Severity: Informational

Throughout the codebase, hard-coded number literals without context or a description are used. These "magic numbers" go against best practices, as they reduce code readability and maintenance. Developers cannot easily understand their use and may make inconsistent changes across the codebase.

Instances of magic numbers are listed below:

- In `gno.land/pkg/gnoland/app.go:85`, the application version is set using a hardcoded string "`dev`" when initializing the `BaseApp`.
- In `gno.land/pkg/gnoland/types.go:59`, the `scanner` used for reading the genesis transactions file is initialized using numbers `1_000_000` and `2_000_000`.

Consequently, using magic numbers leads to maintainability issues and confusion, especially when the app version needs to be updated or dynamically configured.

Recommendation

We recommend using named constants or configuration values instead of directly embedding literals to improve clarity and reduce potential errors in future development.

Status: Reported

This audit was performed on an early version of the protocol.

29. Contracts should implement a two-step ownership transfer

Severity: Informational

In `examples/gno.land/p/demo/ownable/ownable.gno`, the package allows the current owner to execute a one-step ownership transfer.

While this is common practice, it presents a risk for the ownership of the contract to become lost if the owner transfers ownership to an incorrect address. This is especially important for system realms (e.g., `examples/gno.land/r/sys/users/verify.gno`) that influence almost any interaction with the blockchain.

A two-step ownership transfer allows the current owner to propose a new owner. The account proposed as the new owner may then call a function to claim ownership and execute the config update.

Recommendation

We recommend implementing a two-step ownership transfer. The flow can be as follows:

1. The current owner proposes a new owner address that is validated.
2. The new owner account claims ownership, which applies the configuration changes.

Status: Reported

This audit was performed on an early version of the protocol.

30. Outdated comments

Severity: Informational

There are numerous comments in the codebase, which seem to be outdated or not actually implemented.

These comments have been found in the following instances:

- In `gno.land/pkg/sdk/vm/keeper.go:510`, the comment incorrectly suggests that gas charging for executing `MsgCall` has not been implemented, whereas it is already handled.
- In `examples/gno.land/p/demo/avl/node.gno:386`, the comment describes special handling for the `math.MaxInt` value of the `limit` parameter. However, the parameter is never checked against the value and is decremented after any leaf visit, regardless of the `limit` parameter value.

Recommendation

We recommend updating the comments in the abovementioned instances.

Status: Reported

This audit was performed on an early version of the protocol.

31. Unimplemented functionalities in the codebase

Severity: Informational

Several functionalities are currently unimplemented in the codebase:

- In `gno.land/pkg/sdk/vm/handler.go:110` and `116`, the query functions for retrieving the packages and storage store are missing.
- In `gno.land/pkg/sdk/vm/builtins.go:39`, the `TotalCoin` function, which would allow a realm to retrieve a token's total issuance, is not implemented.

Recommendation

We recommend tracking these missing functionalities in an issue management system and implementing them as needed.

Status: Reported

This audit was performed on an early version of the protocol.

32. Panics in the VM affect node execution

Severity: Informational

Panics occurring in the VM can propagate to the node implementation, potentially leading to unexpected behavior:

- In `gno.land/pkg/sdk/vm/builtins.go:25`, `31`, `32`, `35`, `40`, `44`, `47`, `52`, and `55`, panics may arise from code executed within the VM. These panics will leak outside the VM and propagate to `DeliverTx`.
- In `gno.land/pkg/sdk/vm/keeper.go:255`, `295`, `305`, `310`, `435`, `447`, `450`, `454`, and `487`, panics could be handled before reaching `DeliverTx`.
- Special attention should be given to out-of-gas panics that are re-thrown by `doRecover` in `gno.land/pkg/sdk/vm/keeper.go:396`, `488`, `560`, `612`, `754`, and `811`.

Recommendation

We recommend adopting a cleaner approach for error handling without relying on panic recovery in `DeliverTx`.

Status: Reported

This audit was performed on an early version of the protocol.

33. Outstanding `TODO` and `XXX` comments

Severity: Informational

The codebase contains multiple `TODO` and `XXX` comments. This approach to documenting issues and outstanding work has a few disadvantages:

- `TODO` comments can become stale more easily compared to tasks tracked in an issue tracker. As a result, they often do not progress toward resolution.
- These comments highlight weak spots in the codebase, which can be exploited by a malicious party since the code is publicly available.

- These comments must be maintained frequently to remain useful, which is time-consuming. Otherwise, it may mislead future developers or reviewers.

Below is a non-exhaustive list of discovered TODO comments:

- `gno.land/pkg/gnoland/app.go:83`
- `gno.land/pkg/gnoweb/handler.go:93, 226`
- `gno.land/pkg/sdk/vm/handler.go:110, 116`
- `gno.land/pkg/sdk/vm/invariants.go:13`
- `gno.land/pkg/sdk/vm/package.go:19`
- `gno.land/pkg/sdk/vm/keeper.go:3, 109, 359, 510, 711, 712, 768, 769, 828`, and `834`
- `gno.land/pkg/gnoclient/client_txs.go:295`
- `gno.land/cmd/gnoland/start.go:54-58, 374`

Additionally, numerous `XXX` comments are present in the codebase. Most of these comments reference pending tasks, with a total of 89 instances.

Recommendation

We recommend addressing the most critical `TODO` and `XXX` comments and moving the remaining ones into an issue-tracking system.

Status: Reported

This audit was performed on an early version of the protocol.

Potential attack vectors and considerations

Before initiating the audit, the team developed this list of potential attack vectors and key considerations based on the client-provided documentation.

Each area highlights specific risks and considerations to guide the audit process, helping ensure a thorough evaluation of factors that could compromise the system's integrity, performance, or user trust.

Execution

- **Non-determinism**: Ensure restricted or no access to machine-specific states (e.g. timestamps, I/O) that can break deterministic execution. Executing the same code in the same context must always yield the same result.
- **Breaking out of the VM sandbox**: Risk of accessing sensitive resources (e.g. validator signing keys, system databases) or escaping execution boundaries.
- **Re-entrancy**: Repeated or nested calls from within a contract can lead to inconsistent state updates.
- **Infinite loops**: Check if proper limits are in place (storage, memory, compute). Ensure they are adequately enforced to prevent denial-of-service (DoS).
- **Arithmetic issues**: Watch for overflow/underflow in arithmetic operations, including gas accounting and pointer manipulations. Incorrect array bounds calculation can be considered another kind of arithmetic issue.
- **Gas exhaustion**: Ensure gas metering is accurate to avoid DoS from infinite or large computations.
- **Concurrency / Race conditions**: Consider if concurrent operations can cause state inconsistencies.
- **Appropriateness of the gas model**: Validate whether resource usage is properly reflected in gas costs. Underpriced operations can enable spamming or block-stuffing attacks.

State

- **Access control**: Verify that smart contracts cannot modify or access the state of other contracts/accounts without proper permission.

- **Key collisions**: Ensure unique keys and proper indexing to avoid overwriting or unexpected merges.
- **Error handling**: Confirm that failed contract calls or transactions revert state consistently and handle gas charges correctly.
- **State reuse**: Be cautious when accounts or contracts are deleted and re-instantiated; prior state or storage could be misused (similar to selfdestruct issues).
- **Smart contract code integrity**: Confirm code integrity, e.g. via hashing or other verifiable methods.
- **State integrity**: Validate mechanisms to ensure the state is tamper-proof.
- **Consistent state updates**: there must be no partial write, or mixed writes from distinct calls. Contract state updates must be ordered.

Memory management

- **Buffer overflows**: Validate input and indexing operations to prevent out-of-bounds writes.
- **Invalid array access**: arithmetic mistakes in bounds calculation can result in out-of-bounds writes, too.
- **Stack-based issues**: Watch for stack overflows or excessive stack depth (as enforced in EVM). Example of a vulnerability discovered in the CW VM. Another potential vulnerability could be stack underflow.
- **Use-after-free**: Detect references to freed memory that could be reallocated in unexpected ways.
- **Dangling pointers**: Prevent references to invalid memory addresses.
- **Memory leaks**: Avoid indefinite consumption of memory resources.

Contracts

- **Web-related attack vectors (XSS, etc.)**: As the VM may store webpages, consider potential browser-targeted exploits. Use sandboxing to prevent unauthorized code execution or data sharing.
- **Isolation**: Contract execution and storage should remain isolated from other system components.
- **Cross-contract calls**: Ensure robust handling of state changes and re-entrancy across contract boundaries. Contracts can manipulate each other's state, introducing timing

issues, re-entrancy, or delegatecall-like attacks. Cross-contract calls should be ordered.

- **Special instruction behavior**: Environment-dependent opcodes (e.g. ORIGIN, CALLVALUE) in EVM can introduce non-determinism or unexpected flows.
- **Interpreter restrictions**: Disallow or limit operations that bring non-determinism (e.g., random number generators). Example of vulnerability found in CW VM.

Crypto

- **Signature replays**: Make certain each signed transaction cannot be reused in unintended contexts.
- **Weak randomness**: Avoid using libraries or instructions that rely on unpredictable or environment-dependent sources.
- **Hash collisions**: Choose secure hashing algorithms with low collision probability.
- **Key management**: Use strong and securely stored keys to prevent compromise.

Dependencies

- **Malicious/outdated dependencies**: Implement mechanisms to detect malicious updates or usage of deprecated libraries. Enforce version controls and signature checks.

Appendix

1. **Example source code for [“Attackers can craft malicious contracts and trigger a denial of service”]**

The following file shows a simple deeply nested structure that could be abused to perform the attack.

```
package main

func main() {
    // Example of a deeply nested expression:
    var x = ((((((((((1 + 2) + 3) + 4) + 5) + 6) + 7) + 8) + 9) + 10)
    println(x)
}
```

2. **Example scenario for [“Attackers can abuse source code formatting to trigger a denial of service”]**

The following unformatted file:

```
package main;import("fmt";"math");func
main(){if(true){fmt.Println("Hello");fmt.Println(math.Sin(1.2345))};for
i:=0;i<5;i++{fmt.Println(i)}}
```

Is formatted into:

```
package main

import (
	"fmt"
	"math"
)

func main() {
	if true {
		fmt.Println("Hello")
		fmt.Println(math.Sin(1.2345))
	}
```

```
	for i := 0; i < 5; i++ {
		fmt.Println(i)
	}
}
```

The initial file size is approximately 120 bytes, while the formatted version expands to 230 bytes, almost twice the size.

This expansion can be exploited by attackers to perform DoS attacks or store larger files than paid for, leveraging the gas mismatch.

3. Proof of concept for [“Attackers can abuse source code formatting to trigger a denial of service”]

The following snippet can be executed in `gnovm/pkg/gnolang/go2gno_test.go`.

```
func TestParseCheckMemPackage_Formatting(t *testing.T) {
	t.Parallel()

	// Test case with deliberately unformatted code
	unformattedCode := `package ratio;type S struct{A,B,C int;Nested
struct{D,E,F string;Arr []int;Map map[string]string};Arr2D
[][]int;BigMap map[int]map[int]string};var s=S{1,2,3,struct{D,E,F
string;Arr []int;Map
map[string]string}{"dVal","eVal","fVal",[]int{10,20,30},map[string]strin
g{"k1":"v1","k2":"v2"}},[][]int{{1,2},{3,4}},map[int]map[int]string{1:{2
:"two",3:"three"},4:{5:"five"}}};func doStuff(a,b int)int{if
a>b{a-=b;for i:=0;i<5;i++{a++}}else{a+=b};return a};func moreLogic(x
bool,y int,z string)(string,int){if x&&y>10{return z,y*2}else{return
z+"!",y/2}};func bigCheck(arr
[]int)map[int]bool{out:=make(map[int]bool);for i,v:=range arr{if
v%2==0{out[i]=true}else{out[i]=false}};return out}`

	tests := []struct {
		name    string
		format  bool
		input   string
		wantErr bool
	}{
		{
			name:    "no formatting preserves size",
			format:  false,
			input:   unformattedCode,
			wantErr: false,
```

```
		},
		{
			name:   "formatting changes size",
			format: true,
			input:  unformattedCode,
			wantErr: false,
		},
	}

	for _, tt := range tests {
		tt := tt
		t.Run(tt.name, func(t *testing.T) {
			t.Parallel()

			memPkg := &gnovm.MemPackage{
				Name: "test",
				Path: "test",
				Files: []*gnovm.MemFile{
					{
						Name: "test.gno",
						Body: tt.input,
					},
				},
			}

			// Create importer
			imp := &gnoImporter{
				cfg: &types.Config{},
			}

			// Capture original size
			originalSize := len(memPkg.Files[0].Body)

			// Execute parse check
			_, err := imp.parseCheckMemPackage(memPkg, tt.format)

			// Verify error matches expectation
			if (err != nil) != tt.wantErr {
				t.Errorf("parseCheckMemPackage() error = %v,
wantErr %v", err, tt.wantErr)
				return
			}

			// Get new size after formatting
			newSize := len(memPkg.Files[0].Body)
```

```
                sizeDiff := newSize - originalSize

                if sizeDiff > 0 && !tt.format {
                        t.Errorf("Formatting disabled but size change")
                        return
                }

                if tt.format {
                        percentChange := float64(sizeDiff) /
float64(originalSize) * 100
                        t.Logf("File size before formatting: %d bytes",
originalSize)
                        t.Logf("File size after formatting: %d bytes",
newSize)
                        t.Logf("Size difference: %d bytes (%+.2f%%)",
sizeDiff, percentChange)
                }

                // Verify the content is valid Go code
                if tt.format {
                        _, err := parser.ParseFile(token.NewFileSet(),
"test.gno", memPkg.Files[0].Body, parser.ParseComments)
                        if err != nil {
                                t.Errorf("Formatted output is not valid Go
code: %v", err)
                        }
                }
            })
        }
}
```

4. **Proof of concept for [“Potential underflow during AVL node height calculation”]**

The following snippet can be executed in `gno.land/pkg/gnoland/app_test.go`.

```
func TestVMKeeperOverflow(t *testing.T) {
        env := setupTestEnv()
        ctx := env.vmk.MakeGnoTransactionStore(env.ctx)

        // Give "addr1" some gnots.
        addr := crypto.AddressFromPreimage([]byte("addr1"))
        acc := env.acck.NewAccountWithAddress(ctx, addr)
        env.acck.SetAccount(ctx, acc)
```

```
	env.bank.SetCoins(ctx, addr, std.MustParseCoins(coinsString))
	assert.True(t, env.bank.GetCoins(ctx,
addr).IsEqual(std.MustParseCoins(coinsString)))

	// Create test package.
	files := []*gnovm.MemFile{
		{Name: "init.gno", Body: `
package test

func init() {
}

func Echo(msg string) int8 {
	var maxInt8 int8 = 127
	return maxInt8 + 1
}`},
	}
	pkgPath := "gno.land/r/test"
	msg1 := NewMsgAddPackage(addr, pkgPath, files)
	err := env.vmk.AddPackage(ctx, msg1)
	assert.NoError(t, err)

	// Run Echo function.
	coins := std.MustParseCoins(coinsString)
	msg2 := NewMsgCall(addr, coins, pkgPath, "Echo", []string{"hello
world"})
	res, err := env.vmk.Call(ctx, msg2)
	assert.NoError(t, err)
	assert.Equal(t, `("(-128 int8)\n\n", res)
}
```